

Payment Gateway System

Policy Invoice Ingestion, OneInc Integration & SQS-Throttled Processing at 150,000+ Scale

AUTHORS	STATUS	VERSION	SCALE
Platform Engineering	In Review	1.4.0	150,000+ users

§1 Overview & Goals	§6 Security Architecture
§2 System Architecture	§7 Data Models
§3 Component Design	§8 Scalability & SLOs
§4 End-to-End Payment Flow	§9 Rollout Plan
§5 SQS Throttling Strategy	§10 Risks & Open Questions

§1 Overview & Goals

This document describes the design of a payment gateway platform that ingests billing data from upstream Policy Invoicing systems, orchestrates ACH and digital payment processing through the OneInc payment provider, and throttles all outbound payment requests through AWS SQS to sustain throughput at scale for 150,000+ active policyholders.

The system is implemented as a set of FastAPI-based Python microservices and Java billing services, fronted by Nginx as the reverse proxy and load balancer. It is deployed on AWS and adheres to PCI-DSS Level 1 compliance requirements.

Goals

- ✓ Ingest and normalize invoice data from multiple upstream Policy Invoicing systems via REST and event-driven feeds
- ✓ Process ACH and digital payments for 150,000+ policyholders through the OneInc payment gateway
- ✓ Throttle OneInc API calls via SQS to stay within rate limits and prevent cascade failures
- ✓ Maintain end-to-end payment traceability and idempotency for billing operations
- ✓ Achieve PCI-DSS Level 1 compliance across all cardholder and bank data flows
- ✓ Provide retry, dead-letter, and alerting infrastructure for failed payments

Non-Goals

- ✗ This system does not handle premium pricing or underwriting logic
- ✗ Chargebacks and dispute management are handled by OneInc's portal directly
- ✗ First-party card vaulting — tokenization is delegated to OneInc

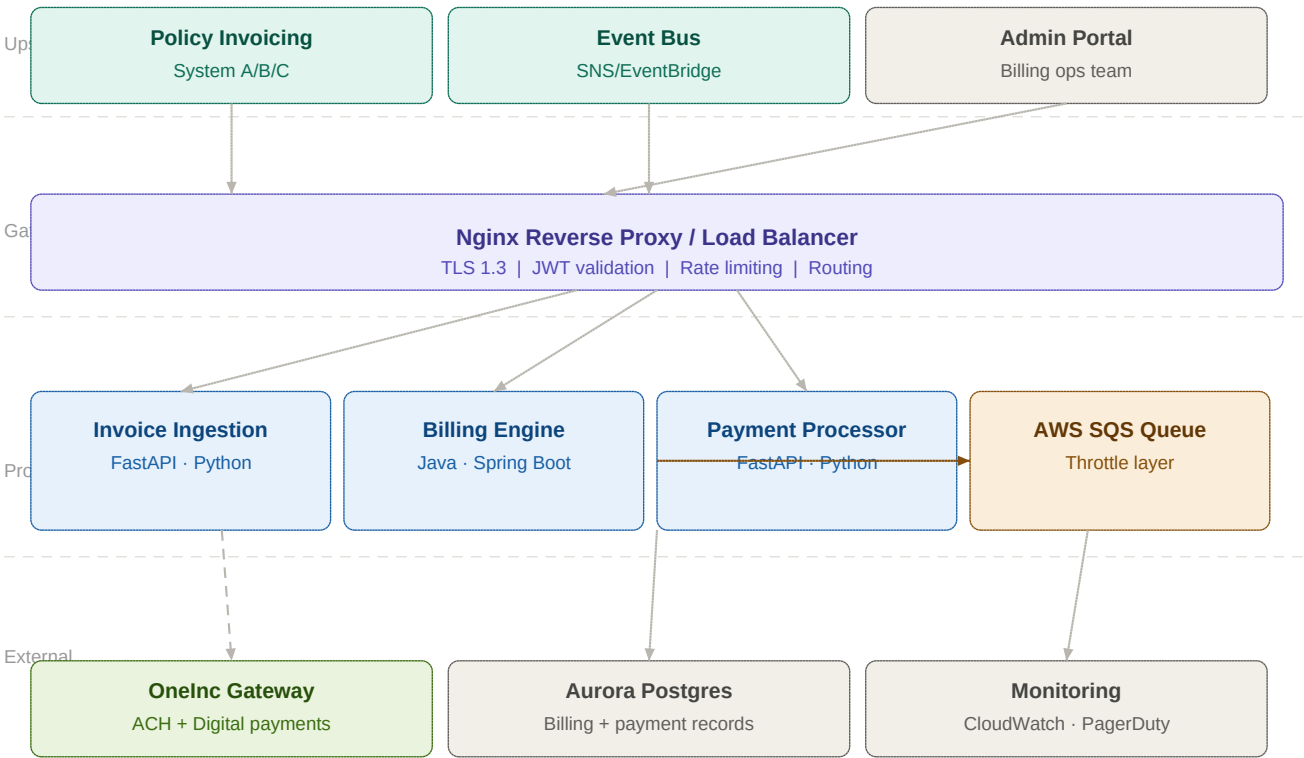
Active Policyholders	Peak TPS (SQS throttled)	Payment Methods	Target Uptime
150,000+	~85 TPS	ACH + Digital	99.95%
Monthly recurring billing	OneInc rate limit ceiling	eCheck, debit, credit	Excl. planned maintenance

§2 System Architecture

The system is organized into four logical tiers: the ingestion layer (Policy invoicing feed), the API gateway layer (Nginx + FastAPI), the processing layer (Java billing engine + SQS worker pool), and the external payment layer (OneInc). Each tier is independently deployable and communicates over defined contracts.

All cross-service communication is asynchronous for payment dispatch and synchronous only for read/query operations. This ensures back-pressure at the SQS layer rather than cascading timeouts into upstream services.

Figure 2.1 — High-Level System Architecture



DESIGN PRINCIPLE

All payment dispatch uses async SQS messaging. Synchronous paths are reserved for reads and status queries only.

§3 Component Design

3.1 Invoice Ingestion Service (Python / FastAPI)

Consumes policy invoice events from SNS/EventBridge and exposes a REST endpoint for synchronous ingestion. Normalizes heterogeneous invoice schemas from Policy Systems A, B, and C into the canonical InvoiceRecord model. Persists to Aurora Postgres with idempotency key enforcement.

Endpoint	Method	Description
/api/v1/invoices/ingest	POST	Synchronous ingestion from upstream push
/api/v1/invoices/{id}	GET	Fetch invoice by ID with status
/api/v1/invoices/batch	POST	Bulk ingestion up to 500 records
/health	GET	Liveness + readiness probe

3.2 Billing Engine (Java / Spring Boot)

The core billing orchestration service responsible for computing amounts due, applying fee schedules, generating payment tasks, and publishing them to the SQS payment queue. Implements Spring Batch for large billing cycle runs across 150,000+ accounts. Exposes internal gRPC endpoints consumed by the Admin Portal service.

3.3 Payment Processor Service (Python / FastAPI)

A pool of SQS consumer workers that dequeue payment tasks and dispatch them to the OneInc REST API. Worker concurrency is bounded by SQS visibility timeout and a configurable semaphore to enforce rate limiting. Implements exponential backoff with jitter for transient failures and writes all outcomes back to the payment ledger in Postgres.

3.4 Nginx Configuration

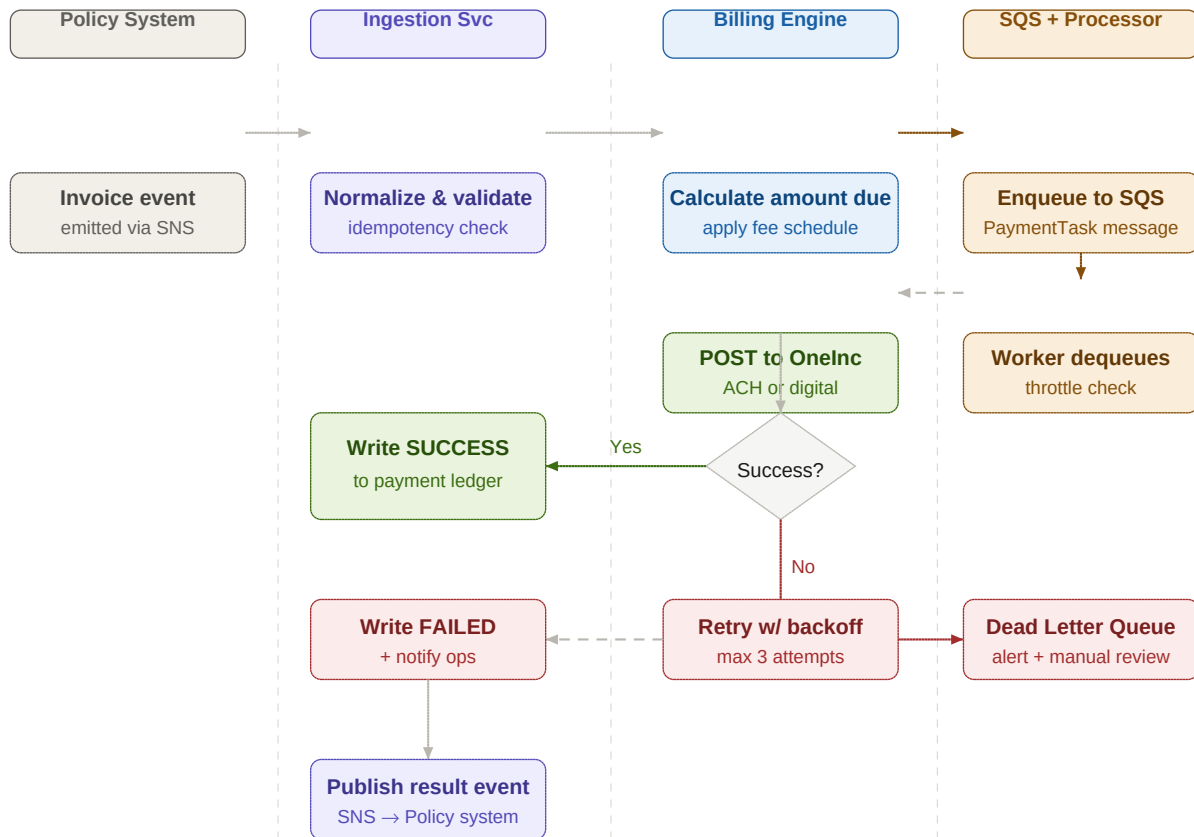
Nginx is configured as the perimeter reverse proxy and L7 load balancer. It handles TLS termination (TLS 1.2+), JWT header forwarding, per-IP rate limiting, and upstream health-check routing across service instances registered in the AWS ECS service discovery DNS.

Directive	Value	Purpose
limit_req_zone	50 r/s per IP	Protects against burst abuse
proxy_read_timeout	30s	Downstream timeout budget
ssl_protocols	TLSv1.2 TLSv1.3	PCI-DSS compliance
upstream keepalive	64 connections	Connection reuse to ECS tasks

§4 End-to-End Payment Flow

The payment lifecycle traverses six stages from invoice event arrival to OneInc confirmation. Each stage writes an audit event to the billing ledger before proceeding. Idempotency is enforced at every transition using a deterministic task_id derived from invoice_id + attempt_number.

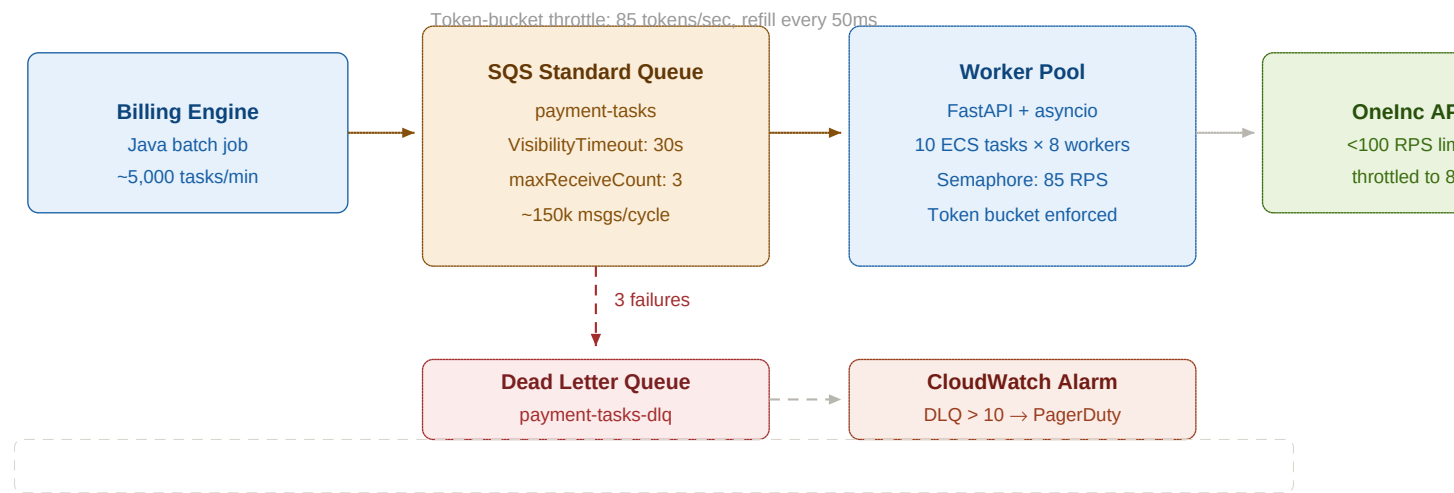
Figure 4.1 — Payment Lifecycle Flow (swimlane)



§5 SQS Throttling Strategy

OneInc enforces a rate limit of approximately 100 requests/second per API credential. With 150,000+ monthly billing events batched into nightly billing windows, unthrottled dispatch would spike to ~1,200 RPS, causing 429 errors and payment failures. The SQS layer smooths this burst into a controlled ~85 RPS sustained flow with 15% safety margin below the OneInc ceiling.

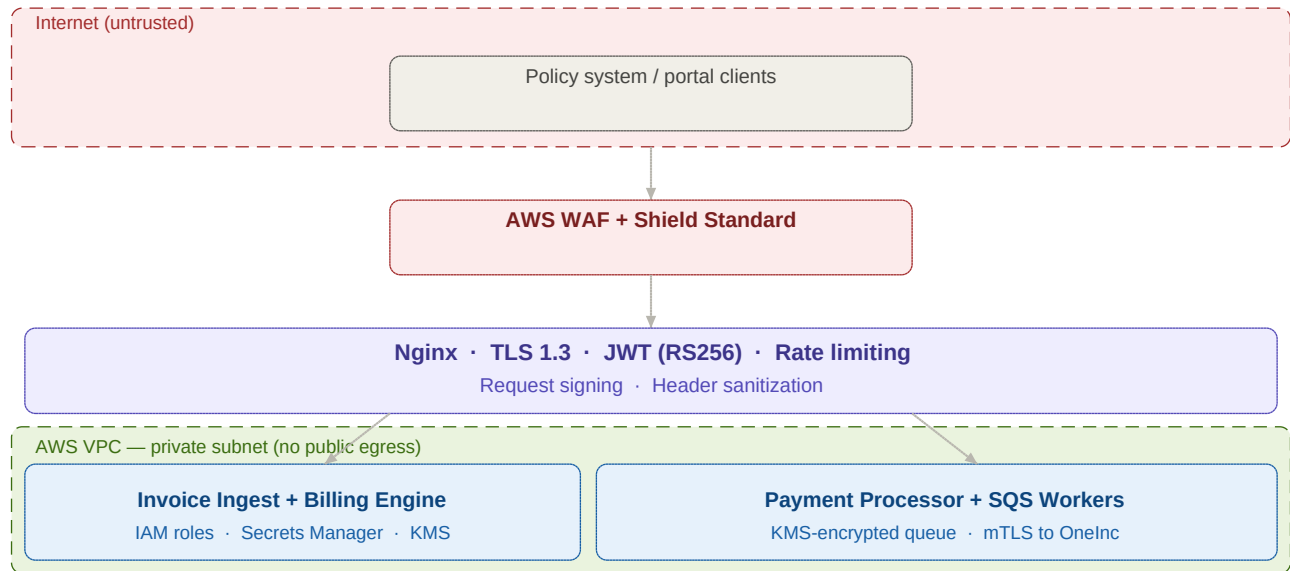
Figure 5.1 — SQS Queue Architecture & Throttle Topology



Parameter	Value	Rationale
Queue type	SQS Standard	At-least-once delivery; idempotency at processor
Visibility timeout	30 seconds	Covers OneInc p99 latency (8s) + overhead margin
Message retention	4 days	Covers weekend + holiday gap before ops review
maxReceiveCount	3	3 attempts before routing to DLQ
Worker concurrency cap	85 RPS (token bucket)	15% below OneInc hard ceiling; burst headroom
ECS task count	10 tasks × 8 workers	80 concurrent slots; auto-scale on SQS depth

§6 Security Architecture

Figure 6.1 — Security Perimeter & Controls



6.1 Authentication & Authorization

Control	Implementation	Standard
API authentication	JWT (RS256), 15-min TTL, JWKS rotation	OAuth 2.0 / RFC 7519
Service-to-service	AWS IAM task roles; no shared keys	Least privilege
OneInc outbound	mTLS client cert + API key in Secrets Manager	PCI-DSS Req 8
Database access	IAM DB authentication; read replicas for reports	PCI-DSS Req 7

6.2 Data Protection

- ✓ All PAN/bank account data is tokenized by OneInc before storage; we store only OneInc token references
- ✓ Aurora Postgres encrypted at rest with AWS KMS CMK; TLS in-transit enforced
- ✓ SQS messages encrypted with KMS; sensitive fields additionally encrypted at application layer (AES-256-GCM)
- ✓ No cardholder data stored in application logs; structured logging masks last-4 only
- ✓ Field-level audit logging in Postgres for all payment_records mutations via trigger

6.3 Network Controls

- ✓ All services in private subnets; NAT Gateway for OneInc egress only (IP allowlisted with OneInc)
- ✓ Security groups enforce least-privilege inbound; no 0.0.0.0/0 on any service port

- ✓ AWS WAF rules: SQLi, XSS, IP reputation, and custom rate-rule per policy-system client
- ✓ VPC Flow Logs enabled; GuardDuty threat detection active on all accounts

PCI-DSS SCOPE

The Payment Processor service, SQS queue, and Aurora payment_records table are in PCI scope. All engineers requiring access must complete annual PCI training; access is provisioned via privileged access workflow with just-in-time approval.

§7 Data Models

7.1 Core Tables (Aurora Postgres)

Table	Key Columns	Description
invoices	id, policy_id, source_system, amount_due, due_date, status, idempotency_key	Normalized invoice from any upstream system
payment_tasks	id, invoice_id, sqs_message_id, payment_method, status, attempt_count	One-per-invoice dispatch record
payment_records	id, task_id, oneinc_token, oneinc_tx_id, amount, status, processed_at	Confirmed OneInc transaction result (PCI scope)
payment_audit_log	id, table_name, record_id, event_type, actor, changed_at	Immutable audit trail via Postgres trigger

7.2 SQS Message Schema (PaymentTask)

Field	Type	Description
task_id	UUID	Idempotency key for OneInc dedup
invoice_id	UUID	Parent invoice reference
payment_method	enum: ACH CARD WALLET	Determines OneInc endpoint
oneinc_token	string	Vaulted payment method token
amount_cents	int64	Amount in cents (no floats)
enqueued_at	ISO 8601	For latency SLA tracking
attempt_number	int	Retry tracking; 0 = first attempt

§8 Scalability & SLOs

API Availability SLO	Payment Success Rate	Ingestion Latency p99	End-to-End Payment SLA
99.95%	>98.5%	<500ms	<4 hours
~22 min error budget/month	Excluding NSF/insufficient funds	Invoice event to Postgres write	Nightly batch fully processed

Auto-scaling Policy

ECS worker tasks scale on two CloudWatch metrics: `ApproximateNumberOfMessagesVisible` (scale out when >5,000 messages) and `NumberOfMessagesSent` rate. Minimum 3 tasks for HA, maximum 20 tasks. Scale-in protected by a 10-minute cooldown to prevent thrashing during billing window ramp-down.

Aurora scales read replicas independently for reporting queries. The Billing Engine uses Spring Batch partitioning to parallelize billing cycle computation across 16 partitions, completing 150k account calculations in under 12 minutes.

§9 Rollout Plan

1 Shadow mode — 2 weeks

Payment tasks enqueued and processed through the new stack in parallel with the legacy billing system. Results compared but no live payments dispatched. Validates data fidelity, SQS depth, and worker throughput.

2 5% canary cutover — 1 week

5% of accounts (targeting low-risk, small-premium policies) routed to new gateway. Monitors payment success rate, DLQ depth, and OneInc confirmation latency. PagerDuty alert threshold tightened to DLQ > 5.

3 Staged ramp — 3 weeks

10% → 25% → 50% → 100% weekly ramp. Each stage requires >98.5% payment success rate and zero P0 incidents in the preceding window. Feature flag controls routing percentage.

4 Legacy decommission — 4 weeks post-full-cutover

Legacy billing dispatch service drained and decommissioned after two full billing cycles confirm stability. Historical payment records migrated to new schema.

§10 Risks & Open Questions

Risk	Severity	Likelihood	Mitigation
OneInc rate limit breach during peak	HIGH	Medium	Token-bucket enforced at 85 RPS; alerting on 429 spike
SQS dedup failure (duplicate payments)	HIGH	Low	Idempotency key as OneInc merchantTransactionId; DB pre-check
Aurora write latency under nightly load	MEDIUM	Medium	PgBouncer connection pooling; batch writes for audit log
OneInc API downtime during billing window	HIGH	Low	SQS 4-day retention; exponential backoff; ops runbook
Policy system schema drift breaks ingestion	MEDIUM	Medium	JSON schema validation at boundary; schema registry; contract tests in CI

Open Questions

#	Question	Owner
Q1	Will OneInc support a higher rate limit (150+ RPS) under a dedicated API credential?	Payments team → OneInc TAM
Q2	Should failed ACH payments trigger an immediate card-on-file retry or require policyholder action?	Product + billing ops
Q3	What is the retention policy for raw invoice events — do we need a replay archive?	Platform Engineering
Q4	Is a FIFO SQS queue required for any policy cohort to guarantee ordered processing?	Billing Engine team

REVIEWERS NEEDED

Security review (§6) from InfoSec · PCI scope sign-off from Compliance · OneInc rate limit confirmation from Vendor team · Load testing sign-off from QA before Phase 2 canary begins.